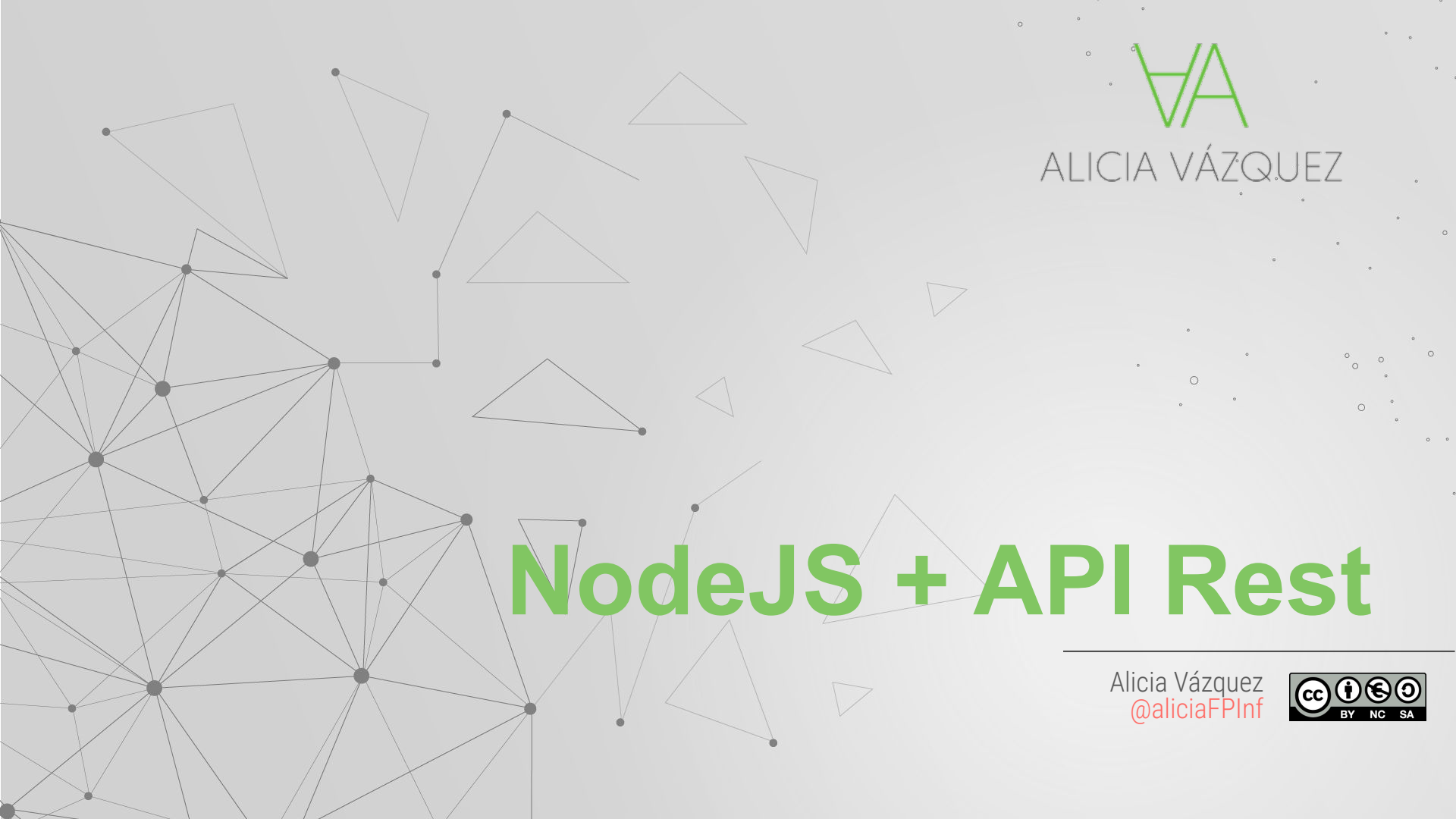


NodeJS + API Rest

Alicia Vázquez
@aliciaFPInf

BY NC SA

01

Node

Què és Node.js

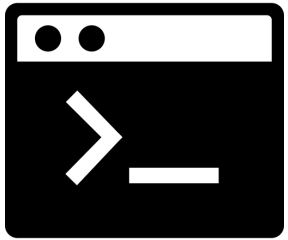
Node.js es un entorno de ejecución (runtime environment en inglés) de JavaScript; es multiplataforma, de código abierto y gratuito.

Permite crear servidores, aplicaciones web, herramientas de línea de comandos y scripts. En definitiva, permite crear casi cualquier tipo de proyecto.

Podemos consultar la documentación oficial y ver la última versión para instalarla en la página web de descarga de Node.js. Download Node.js web site

Instalación

```
# Download and install nvm:
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash

# in lieu of restarting the shell
\. "$HOME/.nvm/nvm.sh"

# Download and install Node.js:
nvm install 24

# Verify the Node.js version:
node -v # Should print "v24.13.1".

# Verify npm version:
npm -v # Should print "11.8.0".
```



Caracteristicas de Node.js

Node.js es un entorno de ejecución de **JavaScript fuera del navegador**. Permite ejecutar JavaScript en el servidor.

A diferencia de otros entornos tradicionales:

- Funciona con un único hilo principal.
- No crea un hilo nuevo por cada petición.
- Utiliza un modelo **asíncrono** y **no bloqueante.**
- Gestiona la concurrencia mediante el **Event Loop** y la librería interna **libuv**.

¿Qué significa que sea no bloqueante?

- Cuando Node recibe una petición:
- Si necesita hacer una operación lenta (leer archivo, consultar base de datos, llamada HTTP…)
- No se queda esperando.
- Delegará esa tarea al sistema (libuv).
- Mientras tanto, sigue atendiendo otras peticiones.
- Por eso puede manejar miles de conexiones con pocos recursos.

02

Proyecto Node

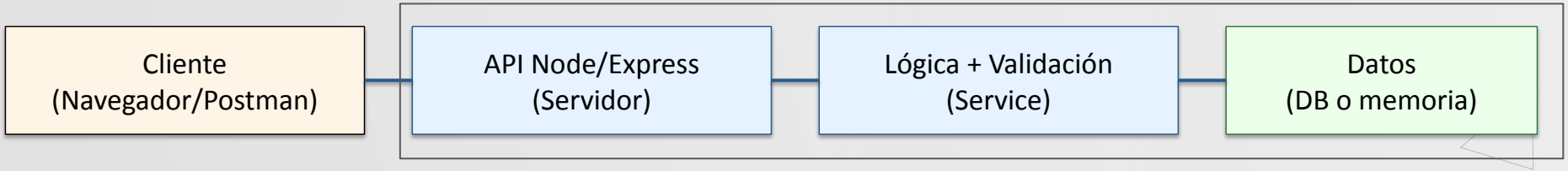
Frontend vs Backend

- **Frontend**: interfaz (HTML/CSS/JS) → lo que ve y usa el usuario.
- **Backend**: lógica + datos → recibe peticiones, procesa, valida y responde.
- Una **API REST** es el “contrato” entre frontend y backend (normalmente JSON).

Protocolo HTTP en API REST (06 Sincrono/Asincrono - API REST)

- **Métodos**: GET (leer), POST (crear), PUT/PATCH (actualizar), DELETE (borrar).
- **Status codes**: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error.
- **Headers**: Content-Type indica el formato (application/json).

Flujo de una petición (Request → Response)



¿Qué es un proyecto Node en backend?

- Una carpeta con código JS que se ejecuta con Node.
- package.json: scripts y dependencias (npm).
- Entrada típica: server.js / index.js / app.js.
- Arranque: **node server.js** o **npm run dev** (con nodemon).

```
const { createServer } = require('node:http');

const hostname = '127.0.0.1';
const port = 3000;

const server = createServer((req, res) => {
 res.statusCode = 200;
 res.setHeader('Content-Type', 'text/plain');
 res.end('Hola Mundo');
});

server.listen(port, hostname, () => {
 console.log(`Server running at
http://${hostname}:${port}/`);
});
```

Copia el código en un fichero **server.js** y abre un terminal
>**node server.js**

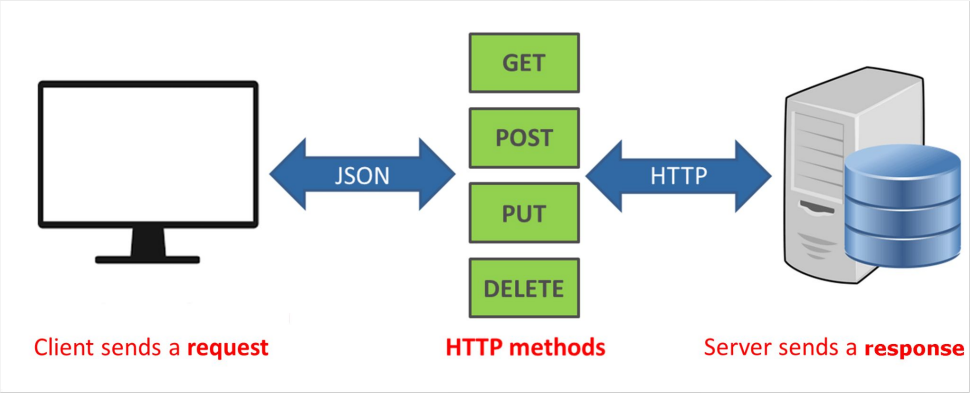
03

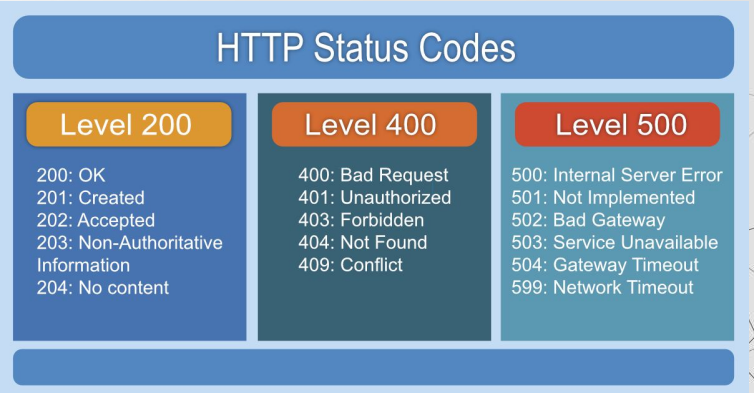
CRUD y API REST

API REST

La **API REST**, interfaz de comunicación que usa el protocolo HTTP para intercambiar datos (normalmente JSON) entre un cliente y un servidor.

- **Métodos**: **GET** (obtener), **POST** (crear), **PUT** (actualizar), **DELETE** (borrar).
- **Status Codes**: 200 (OK), 400 (Errores cliente), 500 (Errores servidor)





CRUD: qué es y para qué sirve

CRUD = Create, Read, Update, Delete.

- Create (POST)
- Read (GET)
- Update (PUT/PATCH)
- Delete (DELETE)

Es la base de gestionar datos: alumnos, productos, reservas, etc.

Ejemplo:

- *GET /api/students → lista de alumnos*
- *GET /api/students/:id → obtener 1 alumno*
- *POST /api/students → crear alumno*
- *PUT /api/students/:id → actualizar alumno*
- *DELETE /api/students/:id → borrar alumno*
- *Extra (opcional): GET /api/students?name=… → filtrar por nombre*

¿Qué es un proyecto Node en backend?

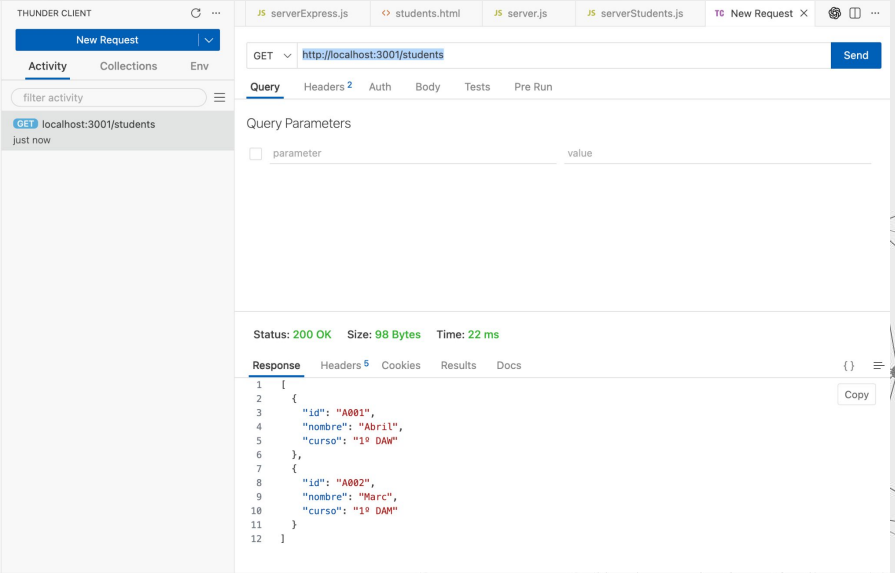
```
const { createServer } = require('node:http');
const hostname = '127.0.0.1';
const port = 3001;

// Datos simulados --> como si fuera lo que nos devuelve la BDD
const students = [
 { id: "A001", nombre: "Abril", curso: "1° DAW" },
 { id: "A002", nombre: "Marc", curso: "1° DAM" }
];

// Método que devuelve la respuesta en formato texto"JSON"
function getStudents(res) {
 res.statusCode = 200;
 //Especifico que el campo relleno es json.
 res.setHeader('Content-Type', 'application/json');
 res.setHeader('Access-Control-Allow-Origin', '*');
 //RECUERDA: Solo podemos enviar texto!!
 res.end(JSON.stringify(students));
}
```

```
const server = createServer((req, res) => {
 console.log(req.method);
 console.log(req.url);
 //ENDPOINT = Metodo + Ruta
 //Si el método es GET y la ruta es "/students" --> Devuelve
el json
 if (req.method === "GET" && req.url === "/students") {
   getStudents(res);
 } else {
   res.statusCode = 404;
   res.setHeader('Content-Type', 'application/json');
   res.end(JSON.stringify({ message: "Not Found" }));
 }
});

server.listen(port, hostname, () => {
 console.log(`Server running at
http://${hostname}:${port}/`);
});
```

Thunder Client

Thunder Client es una extensión de Visual Studio Code que permite hacer peticiones HTTP para probar APIs. Sirve para:

- Probar endpoints GET, POST, PUT, DELETE
- Enviar JSON en el body
- Ver respuestas del servidor
- Ver status codes (200, 404, 500…)
- Probar una API sin necesidad de frontend



Ejercicio 1: CRUD

Haz pruebas con los ficheros que te he dado. Usa Thunder Client y códifica lo siguiente:

En el fichero del servidor:

1. Añade **GET + /students/{$id}**
   Pruebas
   - GET http://127.0.0.1:3001/students/A001 → Devuelve el json con el alumno.
   - GET http://127.0.0.1:3001/students/A999 → Debe devolver 404 y JSON { message: "Not Found" }.
   -

2. Añade **DELETE + /students/{$id}**
   Pruebas
   - DELETE http://127.0.0.1:3001/students/A003 → Devuelve Status 204 (sin body).
   - Lo vuelves a borrar → Debe devolver 404.

Ejercicio 1: CRUD

3. Añade **POST + /students**

   Pruebas

   Body (JSON):

```
{
  "id": "A003",
  "nombre": "Laia",
  "curso": "1º DAW"
}
```

- Status 201 + Respuesta JSON con message Created
- Si repites el mismo id → Debe devolver 409.
- Si falta un campo → Debe devolver 400.

4. Añade **PUT + /students/{$id}**

   Pruebas

- PUT http://127.0.0.1:3001/students/A999 → Debe devolver 404 y JSON { message: "Not Found" }.
- PUT http://127.0.0.1:3001/students/A001 → Devuelve e200 + alumno actualizado

  Body: {

```
  "curso": "2º DAW"
}
```

Ejercicio 2: Resumen

¿Qué es un endpoint?

¿Por qué necesitamos leer el body manualmente en Node HTTP?

¿Qué diferencia hay entre POST y PUT?

¿Por qué DELETE devuelve 204?

04

npm

¿Qué es npm?

npm = **Node Package Manager**

Es:

- El gestor de paquetes de Node
- Una herramienta para instalar librerías
- Un sistema de gestión de dependencias

Con **npm** puedes:

- Instalar librerías (express, prisma, jwt…)
- Ejecutar scripts
- Gestionar versiones
- Publicar paquetes

05

Express

¿Qué es Express?

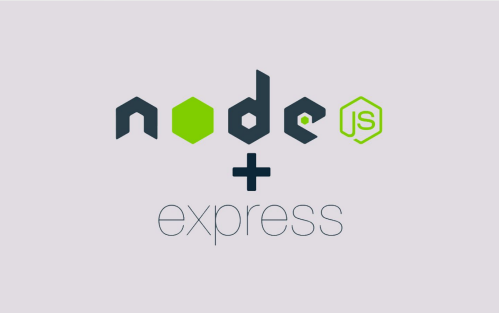
Express es un **framework** para Node.js que facilita la creación de servidores y APIs. Es un servidor HTTP.

- Está construido sobre el módulo http de Node
- Simplifica el manejo de rutas
- Permite trabajar fácilmente con JSON
- Añade estructura al backend
- Express **NO** reemplaza Node
- Express funciona **SOBRE** Node

Con Express puedes:

- Crear APIs REST
- Gestionar usuarios
- Crear microservicios
- Conectar bases de datos

Es la base del backend moderno con Node.



¿Por qué necesitamos Express?

Con **Node** puro:

```
if (req.method === "GET" && req.url === "/students") { ... }
```

Problemas:

- Código repetitivo
- No hay sistema de rutas organizado
- No parsea JSON automáticamente
- No hay middlewares

Express soluciona todo eso, añade:

- Sistema de rutas (**app.get**, **app.post**, etc.)
- Middlewares
- Parsing automático de JSON
- Manejo de errores
- Estructura modular

```
//JS
createServer((req, res) => { ... })

//express
app.get("/students", (req, res) => {
        res.json(students);
});
```

Instalación y uso

Crear proyecto Node

```
npm init -y
```

Instalar Express

```
npm install express
```

Esto crea:

- Carpeta `node_modules`
- Dependencia en `package.json`

Ejecutar servidor

```
npm run dev
```

```
// Importamos Express usando ES Modules
import express from "express";

// Creamos la instancia de la aplicación Express
const app = express();

// Definimos el puerto en una constante para facilitar su
configuración
const PORT = 3000;

// Definimos el endpoint raíz con una arrow function
app.get("/", (req, res) => {
 res.send("Hola Express");
});

// Iniciamos el servidor y escuchamos en el puerto definido
app.listen(PORT, () => {
 console.log(`Servidor corriendo en http://localhost:${PORT}`);
});
```

¿Por qué necesitamos Express?

Con **Node** puro:

```
if (req.method === "GET" && req.url === "/students") { ... }
```

Problemas:

- Código repetitivo
- No hay sistema de rutas organizado
- No parsea JSON automáticamente
- No hay middlewares

Express soluciona todo eso, añade:

- Sistema de rutas (**app.get**, **app.post**, etc.)
- Middlewares
- Parsing automático de JSON
- Manejo de errores
- Estructura modular

```
//JS
createServer((req, res) => { ... })

//express
app.get("/students", (req, res) => {
        res.json(students);
});
```

¿Qué es un Middleware en Express?

Un **middleware** es una función que se ejecuta **entre que llega la petición y antes de que se envíe la respuesta**.

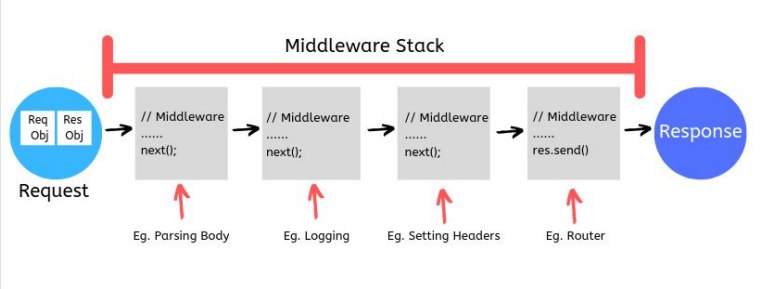
Es decir:

Petición → Middleware → Endpoint → Respuesta

Un middleware puede:

- Leer datos de la petición
- Modificar la petición (`req`)
- Modificar la respuesta (`res`)
- Bloquear la petición
- Pasarla al siguiente middleware (`next()`)

```
//req → información de la petición
//res → resultado
(req, res, next) => {
  // hacer algo
  //función que pasa el control al siguiente middleware
  //sino queda la petición bloqueada.
  next();
}
```



Tipos de Middleware

1. Global

Se ejecuta en TODAS las rutas. Ideal para logs, autenticación o parsear el body.:

```
app.use(miMiddleware);
```

2. Por ruta

Solo se ejecuta en esa ruta:

```
app.get("/students", miMiddleware, (req, res) => {
  res.json([]);
});
```

3. Middleware de errores

Tiene 4 parámetros: (err, req, res, next)

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ message: "Error interno" });
});
```

Solo captura errores si se llama `next(err)` en otro middleware o ruta, no captura errores de forma automática..

Ejemplo

```
// Middleware 1
app.use((req, res, next) => {
 console.log("Soy el primero");
 next(); // Pasa al siguiente
});

// Middleware 2
app.use((req, res, next) => {
 console.log("Soy el segundo");
 next(); // Pasa al siguiente
});

// Ruta final
app.get("/", (req, res) => {
 res.send("Llegué al final");
});
```

Cliente

↓

Middleware 1

↓

Middleware 2

↓

Endpoint

↓

Respuesta

Ejemplo

```
//Middleware 1
app.use((req, res, next) => {
 const error = new Error("Algo salió mal");
 next(error); // Esto salta directamente al manejador de errores
});

//Endpoint
app.get("/", (req, res) => {
 // Esta ruta NUNCA se ejecuta si next(error) fue llamado antes
 res.send("Hola");
});

//Middleware 2 - Solo cuando hay errores.
// Solo llega aquí si alguien llamó next(err)
app.use((err, req, res, next) => {
 console.error(err.message);
 res.status(500).json({ message: "Error interno" });
});
```

Cliente

↓

Middleware 1

↓

Middleware 2 (error)

↓

(NO) Endpoint

↓

Respuesta

Codificación con JS vs codificación con Express

```
function readBody(req) {
 return new Promise((resolve, reject) => {
   let body = "";
   req.on("data", chunk => body += chunk);
   req.on("end", () => {
     try {
       resolve(JSON.parse(body));
     } catch (err) {
       reject(err);
     }
   });
 });
}
```

```
app.use(express.json());
```

Codificación con JS vs codificación con Express

```javascript
// GET /students
if (req.method === "GET" && req.url === "/students") {
  return sendJson(res, 200, students);
}

//GET /students/:id
if (req.method === "GET" && req.url.startsWith("/students/")) {
  // 1. Extraer id de la URL.  Buscar alumno en el array
  const id = req.url.split("/")[2];
  const student = students.find(s => s.id === id);
  // 3. Si no existe → 404
  if (!student) {
    return sendJson(res, 404, { message: "Not Found" });
  }
  // 4. Si existe → devolver 200 + alumno
  return sendJson(res, 200, student);
}
```

```javascript
// GET /students
app.get("/students", (req, res) => {
   res.json(students);
});

// GET /students/:id
app.get("/students/:id", (req, res) => {
   // 1. Extraer id de la URL. Buscar alumno en el array
   const student = students.find(s => s.id === req.params.id);
   // 3. Si no existe → 404
   if (!student) return res.status(404).json({ message: "Not
Found" });
   res.json(student);
});
```

Resumen Express + npm

npm

- Gestiona dependencias
- Crea el proyecto
- Ejecuta scripts

Express

- Simplifica Node
- Organiza endpoints
- Permite crear APIs REST fácilmente

Flujo completo real

1. npm init
2. npm install express
    a. Crear servidor
    b. Definir endpoints
    c. Probar con Postman
3. Conectar frontend

```
//EJEMPLO MINIMO DE EXPRESS
// Importamos Express usando ES Modules
import express from "express";
//const { express } = require('express');

// Creamos la instancia de la aplicación Express
const app = express();

// Definimos el puerto en una constante para facilitar su configuración
const PORT = 3000;

// Definimos el endpoint raíz con una arrow function
app.get("/", (req, res) => {
res.send("Hola Express");
});

// Iniciamos el servidor y escuchamos en el puerto definido
app.listen(PORT, () => {
console.log(`Servidor corriendo en http://localhost${PORT}`);
});
```

Ejercicio 2: Mismo CRUD con Express

Resuelve el mismo problema pero usando **express**.

Haz pruebas con los ficheros que te he dado. Usa Thunder Client y códifica lo siguiente:

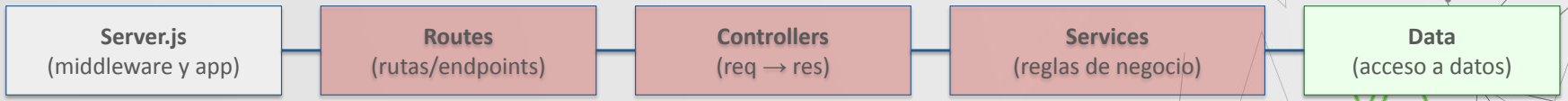
06

Proyecto

Express - Recap

Mejora la fácilidad de programación del codigo nativo: con HTTP nativo, hacer rutas y JSON es más pesado.

- Express añade: routing, middlewares, parseo de JSON, estructura.
- Express NO cambia el motor: sigue siendo Node + Event Loop.
- Arquitectura simple: Distribución por capas, cada capa tiene una responsabilidad. Si mañana cambiamos “memoria” por “BD”, solo cambia Data.



Crear proyecto e instalaciones de dependencias

Creamos un proyecto para poder añadir dependencias. Es en el **package.json** dónde especificamoms toda la info del proyecto y sus necesidades.

La estructura de carpetas es una convención profesional muy consolidada que conviene aprender porque la encontraaremos en los proyectos profesionales.

```
npm init -y
```

```
npm install express
```

```
npm install nodemon --save-dev
```

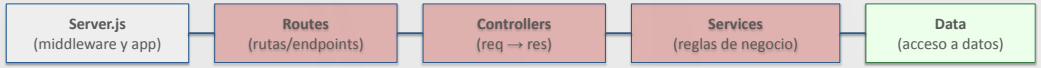
```
npm run dev.
```

```
{
 "name": "servidor-con-npm",
 "version": "1.0.0",
 "description": "",
 "main": "server.js",
 "type": "module",
 "scripts": {
   "test": "echo \"Error: no test specified\" &&
exit 1",
   "dev": "nodemon server.js"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "dependencies": {
   "express": "^5.2.1"
 },
 "devDependencies": {
   "nodemon": "^3.1.14"
 }
}
```

Proyecto: Estructura de carpetas

Cuando creamos un proyecto de Node y hacemos una API Rest es bueno que mantengamos una estructura de carpetas bien estructurada para poder repartir el código.

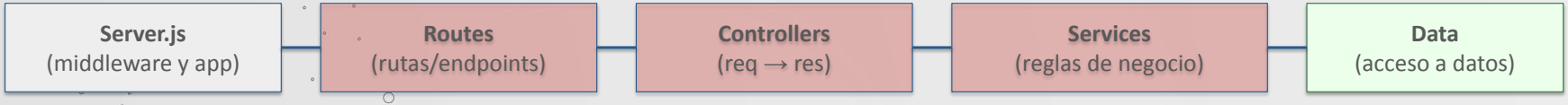
Cada carpeta tendrá un función diferente y por lo tanto una responsabilidad.v



- Fichero **server.js** en la raiz
  - El punto de entrada. Solo arranca el servidor, configura los middlewares globales (cors, express.json) y monta las rutas. No debe contener lógica de negocio.
- Carpeta **routes/**
  - Define los endpoints y los conecta con los controllers. No procesa nada, solo mapea URLs a funciones.
- Carpeta **controllers/**
  - Recibe la **request**, llama al **service**, y devuelve la **response**. Gestiona los códigos de estado HTTP. No accede directamente a los datos.
- Carpeta **services/**
  - Contiene toda la lógica de negocio: validaciones, cálculos, filtros. No sabe nada de HTTP (no conoce req ni res). Hace todas las comprobaciones de los datos que están marcadas por nuestro negocio (ej: valida dni)
- Carpeta **data/**
  - Los datos en memoria o archivos JSON. No tiene lógica.

Para crear todas las carpetas: `mkdir -p routes controllers services data`

Proyecto: Estructura de carpetas



- servidor con NPM
  - controllers
    - students.controller.js
  - data
    - students.js
  - node_modules
  - routes
    - students.routes.js
  - services
    - students.services.js
  - package-lock.json
  - package.json
  - server.js

Server.js

Server.js (middleware y app) — Routes (rutas/endpoints) — Controllers (req → res) — Services (reglas de negocio) — Data (acceso a datos)

```
import express from 'express';
import studentsRouter from
'./routes/students.routes.js';
const app = express();
const PORT = 3001;
// Middlewares globales
app.use(express.json());

// Log mínimo
app.use((req, res, next) => {
 console.log(req.method, req.url);
 next();
});

// Rutas
app.use('/students', studentsRouter);
```

```
// Middleware de errores
app.use((err, req, res, next) => {
 console.error(err.message);
 res.status(500).json({ message: "Error interno" });
});

app.listen(PORT, () => {
 console.log(`Servidor corriendo en
http://localhost:${PORT}/`);
});
```

Routes

Server.js (middleware y app)	Routes (rutas/endpoints)	Controllers (req → res)	Services (reglas de negocio)	Data (acceso a datos)

```
import { Router } from 'express';
import * as studentsController from '../controllers/students.controller.js';

const router = Router();

router.get("/", studentsController.getAll);
router.get("/:id", studentsController.getById);
router.post("/", studentsController.create);
router.put("/:id", studentsController.update);
router.delete("/:id", studentsController.remove);

export default router;
```

Controller

Server.js (middleware y app) — Routes (rutas/endpoints) — Controllers (req → res) — Services (reglas de negocio) — Data (acceso a datos)

```
import * as studentsService from '../services/students.service.js';
export function getAll(req, res) {
 res.json(studentsService.getAll());
}
export function getById(req, res) {
 const student = studentsService.getById(req.params.id);

 if (!student) return res.status(404).json({ message: "Not Found" });
 res.json(student);
}

export function create(req, res) {
 const result = studentsService.create(req.body);

 if (result.error) {
   const status = result.status || 400;
   return res.status(status).json({ message: result.error });
 }

 res.status(201).json({ message: "Created", student: result.data });
}
```

```
export function update(req, res) {
 const updated =
studentsService.update(req.params.id,
req.body);

 if (!updated) return res.status(404).json({
message: "Not Found" });
 res.json(updated);
}

export function remove(req, res) {
 const deleted =
studentsService.remove(req.params.id);

 if (!deleted) return res.status(404).json({
message: "Not Found" });
 res.sendStatus(204);
}
```

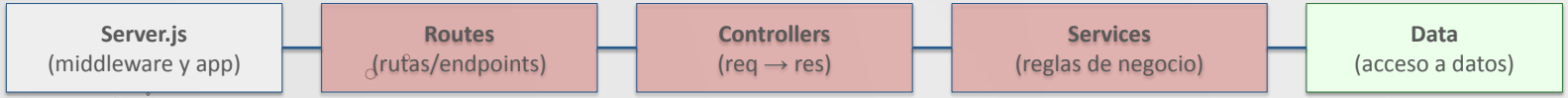
Services

Server.js (middleware y app) — Routes (rutas/endpoints) — Controllers (req → res) — Services (reglas de negocio) — Data (acceso a datos)

```
import { students } from '../data/students.js';

// Valida campos mínimos
function validateStudent(obj) {
 if (!obj || typeof obj !== "object") return "Body inválido";
 if (!obj.id || !obj.nombre || !obj.curso) return "Faltan campos: id, nombre,
curso";
 return null;
}
// Comprueba si el id ya existe
const existsId = (id) => students.some(s => s.id === id);
export function getAll() {
 return students;
}
export function getById(id) {
 return students.find(s => s.id === id);
}
export function create(alumnoNew) {
 const validationMsg = validateStudent(alumnoNew);
 if (validationMsg) return { error: validationMsg };

 if (existsId(alumnoNew.id)) return { error: "id ya existe", status: 409 };

 students.push({ id: alumnoNew.id, nombre: alumnoNew.nombre, curso:
alumnoNew.curso });
 return { data: alumnoNew };
}
```

```
export function update(id, payload) {
 const idx = students.findIndex(s => s.id === id);
 if (idx === -1) return null;

 if (payload && typeof payload === "object") {
   if (payload.nombre !== undefined) students[idx].nombre =
payload.nombre;
   if (payload.curso !== undefined) students[idx].curso =
payload.curso;
 }

 return students[idx];
}

export function remove(id) {
 const before = students.length;
 const filtered = students.filter(s => s.id !== id);

 if (filtered.length === before) return false;

 students.length = 0;
 students.push(...filtered);
 return true;
}
```

Data



- Ahora mismo podemos trabajar directamente con un array de objetos, pero si fuera con una base de datos, el **service** sería prácticamente el mismo.

```
export let students = [
 { id: "A001", nombre: "Abril", curso: "1° DAW" },
 { id: "A002", nombre: "Marc", curso: "1° DAM" }
];
```

```
import db from './db.js';
export function findAll() {
 return db.prepare('SELECT * FROM students').all();
}

export function findById(id) {
 return db.prepare('SELECT * FROM students WHERE id =
?').get(id);
}
```

SERVICE

```
export function getAll() {
 return studentsDb.findAll();
}

export function getById(id) {
 return studentsDb.findById(id);
}
```

Ejercicio 3: Proyecto Alumnos

Crea el proyecto con los diferentes archivos y códigos, verifica que todo funcione correctamente.

Ahora vamos a añadir la tabla notas (**data/notas.js**), a partir de aquí deberás hacer el resto de ficheros. Piensa que hay una relación entre una tabla y otra.

```
export let notas = [
 { id: 1, studentId: "A001", modulo: "M04", nota: 8.5 },
 { id: 2, studentId: "A001", modulo: "M09", nota: 7.0 }
];
// Contador para generar ids automáticos
export let nextId = 3;
```